aspgen WPF-UI Fullstack DDD Developer Guide

A practical, step-by-step guide to building and extending a fullstack (WebApi + WPF-UI desktop) Clean Architecture/DDD application with aspgen.

A practical, step-by-step guide to building and extending a fullstack (WebApi + WPF-UI desktop) Clean Architecture/DDD application with aspgen.

Version 1.0 | Architecture Theta | 04 August 2026

This guide walks through generating a real --app fullstack --backend ddd --theme wpfui application from scratch, command by command, with the actual generated file trees and code. It then covers adding features to an existing app, contrasts this profile against aspgen's other profiles, and closes with end-user tips and a copy-pasteable command cheat sheet. For the underlying architecture rationale see doc/architecture-theta.md; for the full flag/profile matrix across all --app targets see doc/aspgen-generation-decision-guide.md; for the separate Blazor/Renoir DDD profile see doc/aspgen-renoir-developer-guide.md.

1. Architecture & patterns

--app fullstack --backend ddd --theme wpfui combines aspgen's Clean Architecture/DDD Web API with a Prism/DryIoc WPF desktop client styled with the Wpf.Ui ("WPF-UI") Fluent design library. Unlike Renoir, this profile has no bounded-context vocabulary — entities are added flat with add entity, and there is a real HTTP boundary: the desktop client never touches EF Core or the database directly, it talks to the Web API over HttpClient.

Dependency direction across the five generated projects:

```
Desktop (Prism/DryIoc, WPF-UI Fluent controls)
|
v (HttpClient, JSON over HTTP)
WebApi (Minimal API endpoints, thin - only Result <-> Results.* mapping)
|
v
Application (CQRS: Commands/Queries, Handlers, FluentValidation validators)
|
v
Infrastructure (EF Core DbContext, per-entity repository implementations)
|
v
Domain (entities, repository interfaces - no EF/HTTP dependency)
```

Key conventions worth knowing before generating anything:

• **AuditableEntity** — every domain entity inherits CreatedOn/UpdatedOn auditing from AuditableEntity; identity (Id) and mutation (Update(...)) are private-setter/method-only, matching Clean Architecture's "no anemic public setters" rule.

• **CQRS Features folder** — each add entity generates a Features/{Entity}/ folder in Application with one file per operation (Create*Command/Handler/Validator, Update*, Delete*, Get*ById, Get*s, Search*), all implementing the shared IHandler<TRequest, TResponse> returning Result<T>. The WebApi endpoints file is a thin adapter only — it maps Result to Results.Ok/NotFound/ValidationProblem, no business logic.

• **Search + pagination is generated by default.** Every entity gets a Search{Entity}Query/Handler/{Entity}PagedResponse (Application), a /api/{entity}/search endpoint with per-property filter query params (WebApi), and a matching IProductStore.Search(criteria) + {Entity}SearchCriteria/{Entity}PageResult (Desktop) — there's no opt-out flag, this is the standard CRUD shape now.

• **Desktop talks HTTP only, no local EF.** {Entity}Store implements I{Entity}Store via HttpClient (GetFromJsonAsync/PostAsJsonAsync/PutAsJsonAsync/DeleteAsync) against /api/{entity} — it is generated even though the WebApi and Desktop live in the same solution, because a fullstack app is meant to be deployable as two separate processes (API server + desktop client hitting it over the network). The API base URL comes from the ASPGENT_API_URL environment variable (default http://localhost:5000), set once per Prism module ({Entity}Module.RegisterTypes).

• **--theme wpfui swaps XAML chrome only**, not the MVVM shape. Shell.xaml becomes a ui:FluentWindow with ui:TitleBar/ui:NavigationView/ui:ContentDialogHost/ui:SnackbarPresenter; each entity view gains ui:TextBlock headers, ui:Button Appearance="Primary|Danger", and a busy-state ui:ProgressRing. IContentDialogService (destructive-action confirmation) and ISnackbarService (success/error toasts) are registered as singletons in App.xaml.cs only when the theme is wpfui — the plain theme falls back to MessageBox.Show/no toast at all, and both branches keep the same ViewModel command surface.

• **--theme-mode light|dark** (wpfui only, default **light**) only sets App.xaml's <ui:ThemesDictionary Theme="..."> — there's no other startup code to configure; ApplicationThemeManager.GetAppTheme() is read at runtime by the Shell/Settings ViewModels so the UI self-corrects if the user later toggles it live.

• **// aspgen:features / // aspgen:modules / // aspgen:seed markers** — Program.cs (WebApi) and App.xaml.cs (Desktop, ConfigureModuleCatalog) each carry a marker comment where add entity/add module insert their Map*Endpoints()/AddModule<T>() line. Never hand-edit around these; see section 5.

• **add module vs add entity** — a module is a Prism navigation area (views + ViewModel + region registration) with no backend entity attached; add entity (this profile) always creates the full five-layer CRUD slice plus its own Prism module folder under Desktop/Modules/{Entity}/.

2. Build a fullstack DDD WPF-UI app from scratch, step by step

Prerequisites: .NET SDK (see global.json/README for the pinned version). Default database is SQLite (--database sqlite, no separate server needed); pass --database postgres for Postgres instead.

Step 1 — create the project

```
aspgen new WpfUiDemo --app fullstack --backend ddd --theme wpfui --theme-mode light --output
./WpfUiDemo
```

This creates the full project skeleton plus .aspgen/manifest.json:

```
WpfUiDemo/
.aspgen/manifest.json
README.md
WpfUiDemo.sln
src/
Application/
Application.csproj
DependencyInjection.cs
Result.cs
Desktop/
App.xaml, App.xaml.cs
Navigation/NavigationRegistry.cs
Shared/Events/AppEvent.cs
Shell.xaml, Shell.xaml.cs, ShellViewModel.cs
Views/DashboardView.xaml(.cs), DashboardViewModel.cs
Views/SettingsView.xaml(.cs), SettingsViewModel.cs
README.md
WpfUiDemo.Desktop.csproj
Domain/
Domain.csproj
Entities/AuditableEntity.cs
Infrastructure/
DependencyInjection.cs
Infrastructure.csproj
Persistence/AppDbContext.cs
WebApi/
```

```
appsettings.json
Program.cs
WebApi.csproj
```

The manifest records every flag chosen at creation time:

```
{
"project": "WpfUiDemo",
"components": [
"backend:ddd",
"database:sqlite",
"prism-dryioc",
"theme-mode:light",
"theme:wpfui",
"webapi",
"wpf"
]
}
```

Program.cs already has the marker comments waiting for the first entity:

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddOpenApi();
builder.Services.AddApplication();
builder.Services.AddInfrastructure(builder.Configuration);
var app = builder.Build();

app.MapOpenApi();
app.MapScalarApiReference(options => options.WithTitle("WpfUiDemo API"));
app.MapHealthChecks("/health");
// aspgen:seed
// aspgen:features
app.MapGet("/", () => Results.Ok(new { application = "WpfUiDemo" }));
app.Run();
```

Shell.xaml shows the WPF-UI Fluent chrome — FluentWindow, TitleBar, NavigationView, plus the dialog/snackbar hosts that entity ViewModels will use later:

```
<ui:FluentWindow x:Class="WpfUiDemo.Desktop.Shell"
...
ExtendsContentIntoTitleBar="True"
WindowBackdropType="Mica"
prism:ViewModelLocator.AutoWireViewModel="True"
Title="WpfUiDemo">
<Grid>
<Grid.RowDefinitions>
<RowDefinition Height="Auto" />
<RowDefinition Height="*" />
</Grid.RowDefinitions>
<ui:TitleBar Grid.Row="0" Title="WpfUiDemo" Icon="{ui:SymbolIcon DataHistogram24}" />
<Grid Grid.Row="1" Margin="16">
<ui:NavigationView Grid.Column="0" MenuItemsSource="{Binding NavigationItems}"
PaneDisplayMode="Left" PaneTitle="WpfUiDemo">
<ui:NavigationView.PaneFooter>
<StackPanel>
<ui:NavigationViewItem Content="Settings" Icon="{ui:SymbolIcon Settings24}" Command="{Binding
OpenSettingsCommand}" />
<ui:Button Content="{Binding ThemeButtonText}" Icon="{ui:SymbolIcon DarkTheme24}"
Command="{Binding ToggleThemeCommand}" />
</StackPanel>
</ui:NavigationView.PaneFooter>
</ui:NavigationView>
<ContentControl Grid.Column="1" prism:RegionManager.RegionName="MainRegion" />
</Grid>
<ui:ContentDialogHost x:Name="RootContentDialog" Grid.Row="0" Grid.RowSpan="2" />
<ui:SnackbarPresenter x:Name="RootSnackbar" Grid.Row="1" />
</Grid>
</ui:FluentWindow>
```

Step 2 — add an entity

```
aspgen add entity Product name:string price:decimal active:bool category:string --project
./WpfUiDemo
```

This is the single command that produces the whole vertical slice — no separate "aggregate"/"context" steps like Renoir. New files:

```
src/Application/Features/Product/
CreateProductCommand.cs, CreateProductHandler.cs, CreateProductValidator.cs
UpdateProductCommand.cs, UpdateProductHandler.cs, UpdateProductValidator.cs
DeleteProductCommand.cs, DeleteProductHandler.cs
GetProductByIdQuery.cs, GetProductByIdHandler.cs
GetProductsQuery.cs, GetProductsHandler.cs
SearchProductsQuery.cs, SearchProductsHandler.cs
ProductRequest.cs, ProductResponse.cs, ProductPagedResponse.cs
src/Domain/
Entities/Product.cs
Repositories/IProductRepository.cs
src/Infrastructure/Persistence/
Configurations/ProductConfiguration.cs
ProductRepository.cs
src/WebApi/Features/Product/
ProductEndpoints.cs
src/Desktop/Modules/Product/
Models/ProductRow.cs, ProductSearchCriteria.cs, ProductPageResult.cs
Services/IProductStore.cs, ProductStore.cs
ViewModels/ProductViewModel.cs, ProductMenuViewModel.cs, ProductDetailsViewModel.cs
Views/ProductView.xaml(.cs), ProductMenuView.xaml(.cs), ProductDetailsView.xaml(.cs)
ProductModule.cs
```

Domain/Entities/Product.cs — the entity, private-setter properties, Update is the only mutation path:

```
namespace WpfUiDemo.Domain.Entities;

public sealed class Product : AuditableEntity
{
private Product() { }
public long Id { get; private set; }

public Product( string name, decimal price, bool active, string category)
{
Name = name;
Price = price;
Active = active;
Category = category;
}
public string Name { get; private set; } = default!;
public decimal Price { get; private set; } = default!;
public bool Active { get; private set; } = default!;
public string Category { get; private set; } = default!;

public void Update( string name, decimal price, bool active, string category)
{
Name = name;
Price = price;
Active = active;
Category = category;
}
// aspgen:navigation
}
```

Domain/Repositories/IProductRepository.cs — the repository contract, including the search shape:

```
public interface IProductRepository
{
Task<List<Product>> GetAllAsync(CancellationToken cancellationToken = default);
Task<Product?> GetByIdAsync(long id, CancellationToken cancellationToken = default);
Task AddAsync(Product entity, CancellationToken cancellationToken = default);
Task SaveChangesAsync(CancellationToken cancellationToken = default);
Task DeleteAsync(Product entity, CancellationToken cancellationToken = default);
```

```
Task<(List<Product> Items, long TotalCount)> SearchAsync(string? search, string? nameContains,
decimal? priceMin, decimal? priceMax, bool? active, string? categoryContains,
int page, int pageSize, CancellationToken cancellationToken = default);
}
```

Application/Features/Product/CreateProductHandler.cs — CQRS handler, primary-constructor DI, returns Result<T>:

```
public sealed class CreateProductHandler(Domain.Repositories.IProductRepository repository)
: IHandler<CreateProductCommand, ProductResponse>
{
public async Task<Result<ProductResponse>> HandleAsync(CreateProductCommand request,
CancellationToken cancellationToken = default)
{
var entity = new WpfUiDemo.Domain.Entities.Product( request.Name, request.Price, request.Active,
request.Category);
await repository.AddAsync(entity, cancellationToken);
await repository.SaveChangesAsync(cancellationToken);
return Result<ProductResponse>.Success(new ProductResponse(entity.Id, entity.Name, entity.Price,
entity.Active, entity.Category));
}
}
```

WebApi/Features/Product/ProductEndpoints.cs — thin Minimal API adapter, one route per handler:

```
public static class ProductEndpoints
{
public static IEndpointRouteBuilder MapProductEndpoints(this IEndpointRouteBuilder endpoints)
{
var group = endpoints.MapGroup("/api/product");
group.MapGet("/", async (IHandler<GetProductsQuery, IReadOnlyList<ProductResponse>> handler,
CancellationToken ct) =>
{
var result = await handler.HandleAsync(new GetProductsQuery(), ct);
return Results.Ok(result.Value);
});
group.MapGet("/search", async (string? search, string? nameContains, decimal? priceMin, decimal?
priceMax,
bool? active, string? categoryContains, int page, int pageSize,
IHandler<SearchProductsQuery, ProductPagedResponse> handler, CancellationToken ct) =>
{
if (page < 1) page = 1;
if (pageSize < 1) pageSize = 25;
var result = await handler.HandleAsync(new SearchProductsQuery(search, nameContains, priceMin,
priceMax, active, categoryContains, page, pageSize), ct);
return Results.Ok(result.Value);
});
group.MapPost("/", async (ProductRequest request, IHandler<CreateProductCommand,
ProductResponse> handler,
IValidator<CreateProductCommand> validator, CancellationToken ct) =>
{
var command = new CreateProductCommand( request.Name, request.Price, request.Active,
request.Category);
var validation = await validator.ValidateAsync(command, ct);
if (!validation.IsValid) return Results.ValidationProblem(validation.ToDictionary());
var result = await handler.HandleAsync(command, ct);
return Results.Created($"/api/product/{result.Value!.Id}", result.Value);
});
// MapPut, MapDelete follow the same validate-then-handle shape
return group;
}
}
```

Infrastructure/Persistence/ProductRepository.cs — the only layer that talks EF Core directly:

```
public sealed class ProductRepository(AppDbContext database) : IProductRepository
{
public Task<List<Product>> GetAllAsync(CancellationToken cancellationToken = default) =>
database.Products.AsNoTracking().ToListAsync(cancellationToken);
```

```
public Task<Product?> GetByIdAsync(long id, CancellationToken cancellationToken = default) =>
database.Products.FirstOrDefaultAsync(x => x.Id == id, cancellationToken);

public Task AddAsync(Product entity, CancellationToken cancellationToken = default) =>
database.Products.AddAsync(entity, cancellationToken).AsTask();

public async Task<(List<Product> Items, long TotalCount)> SearchAsync(string? search, string?
nameContains,
decimal? priceMin, decimal? priceMax, bool? active, string? categoryContains,
int page, int pageSize, CancellationToken cancellationToken = default)
{
var query = database.Products.AsNoTracking().AsQueryable();
if (!string.IsNullOrWhiteSpace(search)) query = query.Where(x => x.Name.Contains(search) ||
x.Category.Contains(search));
if (priceMin.HasValue) query = query.Where(x => x.Price >= priceMin.Value);
if (active.HasValue) query = query.Where(x => x.Active == active.Value);
var totalCount = await query.LongCountAsync(cancellationToken);
var items = await query.OrderBy(x => x.Id).Skip((page - 1) *
pageSize).Take(pageSize).ToListAsync(cancellationToken);
return (items, totalCount);
}
}
```

Desktop/Modules/Product/Services/ProductStore.cs — the desktop side of the same contract, over HTTP instead of EF:

```
public sealed class ProductStore(HttpClient http) : IProductStore
{
private const string Path = "/api/product";

public IReadOnlyList<ProductRow> GetAll() =>
http.GetFromJsonAsync<List<ProductRow>>(Path).GetAwaiter().GetResult() ?? [];

public ProductPageResult Search(ProductSearchCriteria criteria)
{
var query = new List<string>();
if (!string.IsNullOrWhiteSpace(criteria.Search))
query.Add($"search={Uri.EscapeDataString(criteria.Search)}");
query.Add($"page={criteria.Page}");
query.Add($"pageSize={criteria.PageSize}");
var url = $"{Path}/search?{string.Join("&", query)}";
var result = http.GetFromJsonAsync<ProductPageResult>(url).GetAwaiter().GetResult();
return result ?? new ProductPageResult([], 0, criteria.Page, criteria.PageSize);
}

public ProductRow Save(long editingId, ProductRow value)
{
if (editingId == 0)
{
var response = http.PostAsJsonAsync(Path, value).GetAwaiter().GetResult();
response.EnsureSuccessStatusCode();
return response.Content.ReadFromJsonAsync<ProductRow>().GetAwaiter().GetResult() ?? value;
}
var update = http.PutAsJsonAsync($"{Path}/{editingId}", value).GetAwaiter().GetResult();
update.EnsureSuccessStatusCode();
return value with { Id = editingId };
}

public void Delete(long id)
{
var response = http.DeleteAsync($"{Path}/{id}").GetAwaiter().GetResult();
response.EnsureSuccessStatusCode();
}
}
```

Desktop/Modules/Product/ProductModule.cs — where the HttpClient base address is configured, and the module wires itself into navigation:

```
public sealed class ProductModule : IModule
{
public void RegisterTypes(IContainerRegistry containerRegistry)
{
var apiUrl = Environment.GetEnvironmentVariable("ASPGENT_API_URL") ?? "http://localhost:5000";
containerRegistry.RegisterSingleton<HttpClient>(() => new HttpClient { BaseAddress = new
Uri(apiUrl) });
containerRegistry.RegisterSingleton<Services.IProductStore, Services.ProductStore>();
containerRegistry.RegisterForNavigation<Views.ProductView>();
containerRegistry.RegisterForNavigation<Views.ProductDetailsView>("ProductDetailsView");
}

public void OnInitialized(IContainerProvider containerProvider)
{
var navigation = containerProvider.Resolve<NavigationRegistry>();
navigation.Register("Product", "ProductView", SymbolRegular.Table24);
}
}
```

Desktop/Modules/Product/Views/ProductView.xaml — the WPF-UI themed list page: primary/danger button appearances, a busy ProgressRing, a search box, and a collapsible advanced-filter Expander:

```
<DockPanel>
<StackPanel DockPanel.Dock="Left">
<ui:TextBlock FontTypography="TitleLarge" Text="Product" />
<ui:TextBlock Foreground="{DynamicResource TextFillColorSecondaryBrush}" FontTypography="Body"
Text="Manage Product records." />
</StackPanel>
<StackPanel DockPanel.Dock="Right" Orientation="Horizontal">
<ui:ProgressRing Width="20" Height="20" IsIndeterminate="True" Visibility="{Binding IsBusy,
Converter={StaticResource BoolToVisibility}}" />
<ui:Button Appearance="Primary" Content="New" Icon="{ui:SymbolIcon Add24}" Command="{Binding
NewCommand}" />
<ui:Button Content="Edit" Icon="{ui:SymbolIcon Edit24}" Command="{Binding EditCommand}" />
<ui:Button Appearance="Danger" Content="Delete" Icon="{ui:SymbolIcon Delete24}"
Command="{Binding DeleteCommand}" />
<ui:Button Content="Refresh" Icon="{ui:SymbolIcon ArrowClockwise16}" Command="{Binding
RefreshCommand}" />
</StackPanel>
</DockPanel>
<Expander Header="Advanced filters" IsExpanded="{Binding IsAdvancedFilterExpanded,
Mode=TwoWay}">
<StackPanel Orientation="Horizontal">
<TextBox Text="{Binding Filter.NameContains, UpdateSourceTrigger=PropertyChanged}" />
<TextBox Text="{Binding Filter.PriceMin, UpdateSourceTrigger=PropertyChanged}" />
</StackPanel>
</Expander>
```

Program.cs and App.xaml.cs both gained their marker-anchored registration line:

```
// aspgen:features
app.MapProductEndpoints();
```

```
protected override void ConfigureModuleCatalog(IModuleCatalog moduleCatalog)
{
// aspgen:modules
moduleCatalog.AddModule<WpfUiDemo.Desktop.Modules.Product.ProductModule>();
}
```

Build and run

```
dotnet restore ./WpfUiDemo/WpfUiDemo.sln
dotnet build ./WpfUiDemo/WpfUiDemo.sln
dotnet run --project ./WpfUiDemo/src/WebApi
dotnet run --project ./WpfUiDemo/src/Desktop
```

Start the WebApi project first — the Desktop process needs it reachable at http://localhost:5000 (or whatever ASPGENT_API_URL is set to) before its Prism modules can load data. SQLite is the default Infrastructure provider, so there is no separate database server to stand up; the connection string lives in appsettings.json.

3. Adding new features to an existing app

Adding a second entity later

Nothing is regenerated from scratch — add commands only append. A second entity follows the same shape with a new name:

```
aspgen add entity Category name:string --project ./WpfUiDemo
```

Both Program.cs's // aspgen:features marker and App.xaml.cs's // aspgen:modules marker grow one more line each; Product's files are untouched.

add entity-field — adding a property to an existing entity

```
aspgen add entity-field Product notes:string --project ./WpfUiDemo
```

This patches the entity, every CQRS command/handler that constructs, updates, or maps it into a response — including Create*Handler, Update*Handler, Get*ByIdHandler, Get*sHandler, and Search*sHandler — plus the WebApi request/response records and the Desktop row/view/ViewModel. No manual file editing needed, safe to re-run for additional fields later.

Domain/Entities/Product.cs (constructor + property + Update all gain the new field):

```
- public Product( string name, decimal price, bool active, string category)
+ public Product( string name, decimal price, bool active, string category, string notes)
{
Name = name;
Price = price;
Active = active;
Category = category;
+ Notes = notes;
}
public string Name { get; private set; } = default!;
public decimal Price { get; private set; } = default!;
public bool Active { get; private set; } = default!;
public string Category { get; private set; } = default!;

- public void Update( string name, decimal price, bool active, string category)
+ public void Update( string name, decimal price, bool active, string category, string notes)
{
Name = name;
Price = price;
Active = active;
Category = category;
+ Notes = notes;
}
+ public string Notes { get; private set; } = default!;
```

ProductRequest/ProductResponse (Application) and ProductRow (Desktop) each pick up the matching Notes member, and ProductView.xaml gains a bound TextBox/label pair in the edit form, using the same uiControl mapping (InputText for string) recorded in the manifest.

add module — a navigation area with no backend entity

```
aspgen add module Reporting --project ./WpfUiDemo
```

Use this for a Prism navigation area that isn't backed by a CRUD entity at all (e.g. a dashboard, a settings screen, a reports page you'll hand-write). It creates Desktop/Modules/Reporting/ (view + ViewModel + ReportingModule : IModule) and registers it at the same // aspgen:modules marker add entity uses — but with no Application/Domain/Infrastructure/WebApi files at all, since there's no data behind it yet.

4. Extending: entity vs module vs Renoir vs other backends

aspgen's "add a building block" command depends entirely on which --app/backend profile the project was created with. There is no migration command between profiles — the choice is made once, at aspgen new

time.

Profile (aspgen new flags)	Command to add a feature	What it generates
Fullstack DDD, any theme (--app fullstack --backend ddd)	add entity NAME prop:type...	Domain entity, repository, CQRS CRUD+search, Minimal API endpoints, and a full Prism/Wpf.Ui module — all in one command
Simple Web API/fullstack (--app webapi --simple / --app fullstack --simple)	add entity NAME prop:type...	EF model, DbSet, direct Minimal API CRUD — no CQRS handlers, no repository interface
WPF-only local DDD (--app wpf --backend ddd)	add entity NAME prop:type...	Same DDD layers as fullstack, minus WebApi — Desktop talks to Infrastructure/EF directly, no HttpClient
Any WPF profile, new navigation area with no entity	add module NAME	Prism module: views, ViewModel, navigation registration, nothing else
Renoir / Blazor (--app blazor)	add context then add aggregate/value-object/domain-service/repository/event	DDD building blocks scoped to a bounded context — see doc/aspgen-renoir-developer-guide.md
Any webapi profile, desktop added later	add ui --project PATH	Adds the Prism/DryIoc Desktop project onto an existing server-only project, without regenerating the server

Switching between profiles (e.g. from fullstack DDD to Renoir, or dropping the WPF-only DDD app's HttpClient-less Desktop into a fullstack one) is not supported as an in-place command — see doc/aspgen-generation-decision-guide.md for the full decision table if you're choosing a profile for a new project.

5. Tips, tricks & patching

• **Never hand-edit around // aspgen:features or // aspgen:modules.** Every add entity/add module command inserts its registration line directly at these markers in Program.cs/App.xaml.cs. Moving or deleting a marker comment breaks subsequent add commands' insertion point.

• **The manifest is the source of truth, not the file tree.** .aspgen/manifest.json records every entity, its properties, and every component flag (theme, theme-mode, backend, database) chosen at new time. add commands read it to validate preconditions — don't hand-edit generated files in ways that make them inconsistent with what the manifest thinks exists.

• **Run the WebApi project before the Desktop project.** The Desktop client's HttpClient fails silently (empty grids, no error dialog) if it can't reach the API at startup, since {Entity}Store.GetAll() swallows null into an empty list. Set ASPGENT_API_URL if the API isn't on the default http://localhost:5000.

• **add entity-field is the safe way to add a forgotten property** rather than hand-editing the entity/handlers/endpoints/ViewModel — it keeps all five layers' shapes in sync, including the uiControl mapping.

• **--theme-mode only applies with --theme wpfui.** Passing it with the plain theme (or omitting --theme wpfui entirely) is a validation error — there's no dark/light concept for the plain WPF theme.

• **Corporate NuGet restore workaround.** If dotnet restore/dotnet build fails with NU1301 "No such host is known" behind a corporate proxy or VPN-only NuGet source, restore explicitly from nuget.org first, then build normally:

``bash dotnet restore ./WpfUiDemo/WpfUiDemo.sln -s https://api.nuget.org/v3/index.json dotnet build ./WpfUiDemo/WpfUiDemo.sln ``

• **Avoid Task as an entity name.** It collides with System.Threading.Tasks.Task in generated C# files that have a bare using System.Threading.Tasks; — use TodoItem or similar instead.

• **Troubleshooting a build failure right after add:** check the manifest first — confirm the entity you referenced actually exists there, and confirm the // aspgen:features/// aspgen:modules markers weren't accidentally removed or duplicated. Most post-add compile errors trace back to one of those two things rather than a bug in the generated code itself.

6. Appendix: command cheat sheet

```
# Create the project
aspgen new WpfUiDemo --app fullstack --backend ddd --theme wpfui --theme-mode light --output
./WpfUiDemo

# Add an entity (full CQRS + WebApi + Desktop module in one command)
aspgen add entity Product name:string price:decimal active:bool category:string --project
./WpfUiDemo

# Add a field to an existing entity later
aspgen add entity-field Product notes:string --project ./WpfUiDemo

# Add a navigation area with no backend entity
aspgen add module Reporting --project ./WpfUiDemo

# Build and run (API first, then desktop)
dotnet restore ./WpfUiDemo/WpfUiDemo.sln
dotnet build ./WpfUiDemo/WpfUiDemo.sln
dotnet run --project ./WpfUiDemo/src/WebApi
dotnet run --project ./WpfUiDemo/src/Desktop

# Corporate NuGet restore workaround
dotnet restore ./WpfUiDemo/WpfUiDemo.sln -s https://api.nuget.org/v3/index.json
```